

ML Lab Programs - CDAC PGCP-BDA

Executable Jupyter / VS Code Code for Practical Machine Learning Labs

Prepared for Anmol Gangwar

June 2026

Contents

1	How to use this PDF	2
2	Lab Index	2
3	Lab 1: requirements.txt - Install packages	2
4	Lab 2: Common setup helpers	2
5	Lab 3: Data loading, EDA, nulls, correlation	3
6	Lab 4: Mall Customers: K-Means, WSS, best K, PCA	4
7	Lab 5: Classification: Logistic, SVM, Random Forest	6
8	Lab 6: KNN algorithm	8
9	Lab 7: Regression: Linear, Ridge, Lasso, Polynomial, ElasticNet	9
10	Lab 8: Air Quality: AQI regression and SVM classification	10
11	Lab 9: Time Series: ACF, PACF, smoothing, ARMA, ARIMA	12
12	Lab 10: Recommendation Systems: CF, factorization, metrics	14
13	Lab 11: TensorFlow/Keras: activations and single-layer NN	16
14	Lab 12: Autoencoder reconstruction	18
15	Lab 13: Backprop, weight update, Early Stopping, Dropout, L1/L2	19
16	Lab 14: Batch, Mini-batch, SGD, Momentum, Adam, gradients	20
17	Lab 15: PyTorch CNN and data augmentation	22
18	Lab 16: Transfer Learning with Inception Network	24
19	Lab 17: YOLO object detection with Ultralytics	25
20	Lab 18: Bonus: RNN, LSTM, GRU sequence forecasting	26
21	Lab 19: Bonus: BERT text classification	27
22	Lab 20: End-to-end sklearn pipeline and model saving	28

1 How to use this PDF

All programs below are written as executable Python code. In VS Code, save a section as `.py` and run it cell-by-cell using `# %%`. In Jupyter Notebook, copy each `# %%` block into separate cells. Dataset paths are configurable. When a syllabus dataset is not present, most programs create a small demo dataset so the code still runs.

2 Lab Index

1. requirements.txt - Install packages
2. Common setup helpers
3. Data loading, EDA, nulls, correlation
4. Mall Customers: K-Means, WSS, best K, PCA
5. Classification: Logistic, SVM, Random Forest
6. KNN algorithm
7. Regression: Linear, Ridge, Lasso, Polynomial, ElasticNet
8. Air Quality: AQI regression and SVM classification
9. Time Series: ACF, PACF, smoothing, ARMA, ARIMA
10. Recommendation Systems: CF, factorization, metrics
11. TensorFlow/Keras: activations and single-layer NN
12. Autoencoder reconstruction
13. Backprop, weight update, Early Stopping, Dropout, L1/L2
14. Batch, Mini-batch, SGD, Momentum, Adam, gradients
15. PyTorch CNN and data augmentation
16. Transfer Learning with Inception Network
17. YOLO object detection with Ultralytics
18. Bonus: RNN, LSTM, GRU sequence forecasting
19. Bonus: BERT text classification
20. End-to-end sklearn pipeline and model saving

3 Lab 1: requirements.txt - Install packages

```
# Save as requirements.txt and run:
# python -m pip install -r requirements.txt

numpy
pandas
matplotlib
scikit-learn
scipy
statsmodels
joblib
seaborn

# Deep learning labs - install only if required:
tensorflow
keras
torch
torchvision
ultralytics
transformers
datasets

# Optional for notebooks:
jupyter
ipykernel
```

4 Lab 2: Common setup helpers

```
# %%
# Common setup used in multiple labs.
# In VS Code, each # %% becomes a runnable cell.

import os
import warnings
```

```
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.preprocessing import StandardScaler, MinMaxScaler, OneHotEncoder, LabelEncoder, PolynomialFeatures
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix,
    roc_auc_score,
    log_loss,
    mean_absolute_error,
    mean_squared_error,
    r2_score,
)

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

def rmse(y_true, y_pred):
    return np.sqrt(mean_squared_error(y_true, y_pred))

def print_section(title):
    print("\n" + "=" * 80)
    print(title)
    print("=" * 80)

def basic_eda(df, target_col=None):
    print_section("BASIC EDA")
    print("First 5 rows:")
    print(df.head())
    print("\nLast 5 rows:")
    print(df.tail())
    print("\nShape:", df.shape)
    print("\nInfo:")
    print(df.info())
    print("\nDescribe numeric:")
    print(df.describe())
    print("\nMissing values:")
    print(df.isna().sum())
    numeric_df = df.select_dtypes(include=[np.number])
    if numeric_df.shape[1] > 1:
        print("\nCorrelation matrix:")
        print(numeric_df.corr().round(3))
    if target_col and target_col in df.columns:
        print("\nTarget distribution:")
        print(df[target_col].value_counts(dropna=False))
```

5 Lab 3: Data loading, EDA, nulls, correlation

```
# %%
# Lab target:
# 1. Load any dataset.
# 2. Print first five and last five rows.
# 3. Check shape, info, describe, missing values.
# 4. Check correlation.
# 5. Do a basic train-test split.

import os
import numpy as np
import pandas as pd

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

# Change this path if your CSV is available.
CSV_PATH = "sample_dataset.csv"

if os.path.exists(CSV_PATH):
    df = pd.read_csv(CSV_PATH)
    target_col = df.columns[-1]
else:
```

```

data = load_breast_cancer(as_frame=True)
df = data.frame.copy()
target_col = "target"

print("First 5 rows:")
print(df.head())

print("\nLast 5 rows:")
print(df.tail())

print("\nShape:", df.shape)

print("\nInfo:")
print(df.info())

print("\nDescribe:")
print(df.describe())

print("\nMissing values:")
print(df.isna().sum())

numeric_df = df.select_dtypes(include=[np.number])
print("\nCorrelation matrix:")
print(numeric_df.corr().round(3))

if target_col in df.columns:
    X = df.drop(columns=[target_col])
    y = df[target_col]
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y if y.nunique() <= 20 else None
    )
    print("\nTrain shape:", X_train.shape, y_train.shape)
    print("Test shape:", X_test.shape, y_test.shape)

```

6 Lab 4: Mall Customers: K-Means, WSS, best K, PCA

```

# %%
# Lab target:
# Download Mall_Customers.csv from Kaggle.
# (a) Form number of clusters according to observation.
# (b) Get WSS value for each cluster.
# (c) Find best K value using Elbow and Silhouette.
# Extra syllabus coverage: scaling, distance effect, PCA, random projection, cluster profiling.

import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.cluster import KMeans, AgglomerativeClustering, DBSCAN
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score, davies_bouldin_score, calinski_harabasz_score
from sklearn.decomposition import PCA
from sklearn.random_projection import GaussianRandomProjection
from scipy.cluster.hierarchy import dendrogram, linkage

# %%
# Load dataset or create fallback demo data.
possible_files = ["Mall_Customers.csv", "mall_customers.csv", "Mall_Customers_Dataset.csv"]
csv_path = next((p for p in possible_files if os.path.exists(p)), None)

if csv_path:
    df = pd.read_csv(csv_path)
else:
    rng = np.random.default_rng(42)
    n = 220
    df = pd.DataFrame({
        "CustomerID": np.arange(1, n + 1),
        "Gender": rng.choice(["Male", "Female"], size=n),
        "Age": rng.integers(18, 70, size=n),
        "Annual Income (k$)": np.concatenate([
            rng.normal(35, 8, 70),
            rng.normal(65, 10, 80),
            rng.normal(95, 12, 70),
        ]).round(1),
        "Spending Score (1-100)": np.concatenate([
            rng.normal(30, 10, 70),
            rng.normal(60, 10, 80),
            rng.normal(82, 8, 70),
        ]).clip(1, 100).round(1),
    })

print(df.head())
print(df.info())

```

```

print(df.describe())

# %%
# Select features. The classic Mall Customers lab mostly uses Income and Spending Score.
feature_cols = []
for col in ["Annual Income (k$)", "Spending Score (1-100)", "Age"]:
    if col in df.columns:
        feature_cols.append(col)

if not feature_cols:
    feature_cols = df.select_dtypes(include=[np.number]).columns.tolist()[:3]

X = df[feature_cols].copy()
print("Features used:", feature_cols)

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# %%
# WSS/Inertia for K = 1 to 10.
wss = []
silhouette = []
k_values = range(1, 11)

for k in k_values:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = km.fit_predict(X_scaled)
    wss.append(km.inertia_)
    if k >= 2:
        silhouette.append(silhouette_score(X_scaled, labels))
    else:
        silhouette.append(np.nan)

wss_df = pd.DataFrame({"K": list(k_values), "WSS_Inertia": wss, "Silhouette": silhouette})
print(wss_df)

plt.figure(figsize=(7, 4))
plt.plot(list(k_values), wss, marker="o")
plt.xlabel("Number of clusters K")
plt.ylabel("WSS / Inertia")
plt.title("Elbow Method")
plt.grid(True)
plt.show()

plt.figure(figsize=(7, 4))
plt.plot(list(k_values), silhouette, marker="o")
plt.xlabel("Number of clusters K")
plt.ylabel("Silhouette Score")
plt.title("Silhouette Method")
plt.grid(True)
plt.show()

# %%
# Simple automatic best K using maximum silhouette score for K >= 2.
best_k = int(wss_df.loc[wss_df["Silhouette"].idxmax(), "K"])
print("Best K by Silhouette:", best_k)
print("For the classic Mall Customers dataset, K=5 is often selected from the elbow plot.")

kmeans = KMeans(n_clusters=best_k, random_state=42, n_init=10)
df["Cluster"] = kmeans.fit_predict(X_scaled)

print("\nCluster counts:")
print(df["Cluster"].value_counts().sort_index())

print("\nCluster profile:")
print(df.groupby("Cluster")[feature_cols].mean().round(2))

print("\nCluster evaluation:")
print("Silhouette:", silhouette_score(X_scaled, df["Cluster"]).round(3))
print("Davies-Bouldin:", davies_bouldin_score(X_scaled, df["Cluster"]).round(3))
print("Calinski-Harabasz:", calinski_harabasz_score(X_scaled, df["Cluster"]).round(3))

# %%
# Visualize clusters using first two selected features.
plt.figure(figsize=(7, 5))
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=df["Cluster"], s=40)
plt.xlabel(feature_cols[0] + " scaled")
plt.ylabel(feature_cols[1] + " scaled")
plt.title("K-Means Clusters")
plt.show()

# %%
# Hierarchical clustering and dendrogram.
linked = linkage(X_scaled, method="ward")
plt.figure(figsize=(10, 5))
dendrogram(linked, truncate_mode="lastp", p=30)
plt.title("Hierarchical Clustering Dendrogram")
plt.xlabel("Clustered samples")

```

```

plt.ylabel("Distance")
plt.show()

agg = AgglomerativeClustering(n_clusters=best_k, linkage="ward")
df["AgglomerativeCluster"] = agg.fit_predict(X_scaled)
print(df[["Cluster", "AgglomerativeCluster"]].head())

# %%
# PCA as representation: reduce to 2D for visualization.
pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X_scaled)
print("PCA explained variance ratio:", pca.explained_variance_ratio_.round(3))

plt.figure(figsize=(7, 5))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=df["Cluster"], s=40)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("PCA Representation of Clusters")
plt.show()

# %%
# Random projection as representation.
rp = GaussianRandomProjection(n_components=2, random_state=42)
X_rp = rp.fit_transform(X_scaled)

plt.figure(figsize=(7, 5))
plt.scatter(X_rp[:, 0], X_rp[:, 1], c=df["Cluster"], s=40)
plt.xlabel("Random Projection 1")
plt.ylabel("Random Projection 2")
plt.title("Random Projection Representation")
plt.show()

# %%
# DBSCAN example for density-based clustering.
dbscan = DBSCAN(eps=0.8, min_samples=5)
df["DBSCAN_Cluster"] = dbscan.fit_predict(X_scaled)
print("DBSCAN labels (-1 means noise):")
print(df["DBSCAN_Cluster"].value_counts().sort_index())

```

7 Lab 5: Classification: Logistic, SVM, Random Forest

```

# %%
# Lab target:
# Implement Random Forest, SVM, Logistic Regression classification algorithms.
# Check classification_report and F1-score for all three algorithms.
# Extra metrics: Accuracy, Precision, Recall, ROC-AUC, LogLoss.

import os
import numpy as np
import pandas as pd

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV, cross_val_score
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    classification_report, confusion_matrix, roc_auc_score, log_loss
)

# %%
# Load local dataset or sklearn fallback.
CSV_PATH = "classification_dataset.csv"
TARGET_COL = None # Set target column name if using your own CSV.

if os.path.exists(CSV_PATH):
    df = pd.read_csv(CSV_PATH)
    target_col = TARGET_COL or df.columns[-1]
else:
    data = load_breast_cancer(as_frame=True)
    df = data.frame.copy()
    target_col = "target"

print(df.head())
print("Shape:", df.shape)
print("Target:", target_col)

X = df.drop(columns=[target_col])
y = df[target_col]

```

```

# Encode target if it is text.
if y.dtype == "object":
    from sklearn.preprocessing import LabelEncoder
    le = LabelEncoder()
    y = le.fit_transform(y)

num_cols = X.select_dtypes(include=[np.number]).columns.tolist()
cat_cols = X.select_dtypes(exclude=[np.number]).columns.tolist()

numeric_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
])

categorical_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore")),
])

preprocess = ColumnTransformer([
    ("num", numeric_pipe, num_cols),
    ("cat", categorical_pipe, cat_cols),
])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# %%
models = {
    "Logistic Regression": LogisticRegression(max_iter=1000, random_state=42),
    "SVM RBF": SVC(kernel="rbf", probability=True, random_state=42),
    "Random Forest": RandomForestClassifier(n_estimators=200, random_state=42),
}

results = []

for name, model in models.items():
    pipe = Pipeline([
        ("preprocess", preprocess),
        ("model", model),
    ])

    pipe.fit(X_train, y_train)
    y_pred = pipe.predict(X_test)

    if hasattr(pipe.named_steps["model"], "predict_proba"):
        y_proba = pipe.predict_proba(X_test)
    else:
        y_proba = None

    print("\n" + "=" * 80)
    print(name)
    print("=" * 80)
    print("Confusion Matrix:")
    print(confusion_matrix(y_test, y_pred))
    print("\nClassification Report:")
    print(classification_report(y_test, y_pred))

    row = {
        "Model": name,
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision_macro": precision_score(y_test, y_pred, average="macro"),
        "Recall_macro": recall_score(y_test, y_pred, average="macro"),
        "F1_macro": f1_score(y_test, y_pred, average="macro"),
    }

    if y_proba is not None:
        try:
            if len(np.unique(y)) == 2:
                row["ROC_AUC"] = roc_auc_score(y_test, y_proba[:, 1])
            else:
                row["ROC_AUC"] = roc_auc_score(y_test, y_proba, multi_class="ovr")
            row["LogLoss"] = log_loss(y_test, y_proba)
        except Exception as e:
            row["ROC_AUC"] = np.nan
            row["LogLoss"] = np.nan

    results.append(row)

results_df = pd.DataFrame(results).sort_values(by="F1_macro", ascending=False)
print("\nFinal comparison:")
print(results_df.round(4))

# %%
# Optional tuning for SVM.
svm_pipe = Pipeline([
    ("preprocess", preprocess),

```

```

    ("model", SVC(probability=True, random_state=42)),
])

params = {
    "model__kernel": ["linear", "rbf"],
    "model__C": [0.1, 1, 10],
    "model__gamma": ["scale", "auto"],
}

grid = GridSearchCV(svm_pipe, params, cv=5, scoring="f1_macro")
grid.fit(X_train, y_train)
print("Best SVM params:", grid.best_params_)
print("Best CV F1:", grid.best_score_)

```

8 Lab 6: KNN algorithm

```

# %%
# Lab target:
# Implement K-Nearest Neighbors algorithm.
# Show why scaling and K value matter.

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix, classification_report

# %%
data = load_iris(as_frame=True)
df = data.frame.copy()
print(df.head())

X = df.drop(columns=["target"])
y = df["target"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# %%
# Train KNN with scaling.
knn_pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("knn", KNeighborsClassifier(n_neighbors=5)),
])

knn_pipe.fit(X_train, y_train)
y_pred = knn_pipe.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("F1 macro:", f1_score(y_test, y_pred, average="macro"))
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("Classification Report:")
print(classification_report(y_test, y_pred, target_names=data.target_names))

# %%
# Compare different K values.
k_scores = []
for k in range(1, 31):
    pipe = Pipeline([
        ("scaler", StandardScaler()),
        ("knn", KNeighborsClassifier(n_neighbors=k)),
    ])
    pipe.fit(X_train, y_train)
    pred = pipe.predict(X_test)
    k_scores.append({
        "K": k,
        "Accuracy": accuracy_score(y_test, pred),
        "F1_macro": f1_score(y_test, pred, average="macro"),
    })

k_df = pd.DataFrame(k_scores)
print(k_df.head())

plt.figure(figsize=(8, 4))
plt.plot(k_df["K"], k_df["Accuracy"], marker="o", label="Accuracy")
plt.plot(k_df["K"], k_df["F1_macro"], marker="o", label="F1 macro")
plt.xlabel("K")

```



```

plt.ylabel("Score")
plt.title("KNN score for different K values")
plt.legend()
plt.grid(True)
plt.show()

# %%
# GridSearchCV for best K and distance metric.
param_grid = {
    "knn__n_neighbors": list(range(1, 31)),
    "knn__weights": ["uniform", "distance"],
    "knn__metric": ["euclidean", "manhattan", "minkowski"],
}

grid = GridSearchCV(knn_pipe, param_grid, cv=5, scoring="f1_macro")
grid.fit(X_train, y_train)

print("Best params:", grid.best_params_)
print("Best CV score:", grid.best_score_)
print("Test score:", grid.score(X_test, y_test))

```

9 Lab 7: Regression: Linear, Ridge, Lasso, Polynomial, ElasticNet

```

# %%
# Lab target:
# Download/use a dataset, perform Linear, Ridge, Lasso, Polynomial, ElasticNet regression.
# Check MAE, MSE, RMSE, R2 and write conclusion.
# Note: F1-score is a classification metric, not a pure regression metric.

import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.preprocessing import StandardScaler, OneHotEncoder, PolynomialFeatures
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# %%
def regression_metrics(y_true, y_pred):
    return {
        "MAE": mean_absolute_error(y_true, y_pred),
        "MSE": mean_squared_error(y_true, y_pred),
        "RMSE": np.sqrt(mean_squared_error(y_true, y_pred)),
        "R2": r2_score(y_true, y_pred),
    }

# %%
CSV_PATH = "regression_dataset.csv"
TARGET_COL = None # set manually if using your CSV

if os.path.exists(CSV_PATH):
    df = pd.read_csv(CSV_PATH)
    target_col = TARGET_COL or df.columns[-1]
else:
    data = load_diabetes(as_frame=True)
    df = data.frame.copy()
    target_col = "target"

print(df.head())
print("Shape:", df.shape)
print("Target:", target_col)

X = df.drop(columns=[target_col])
y = df[target_col]

num_cols = X.select_dtypes(include=[np.number]).columns.tolist()
cat_cols = X.select_dtypes(exclude=[np.number]).columns.tolist()

numeric_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
])

categorical_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore")),
])

```

```

preprocess = ColumnTransformer([
    ("num", numeric_pipe, num_cols),
    ("cat", categorical_pipe, cat_cols),
])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# %%
models = {
    "Linear Regression": LinearRegression(),
    "Ridge Regression": Ridge(alpha=1.0, random_state=42),
    "Lasso Regression": Lasso(alpha=0.01, random_state=42, max_iter=10000),
    "ElasticNet Regression": ElasticNet(alpha=0.01, l1_ratio=0.5, random_state=42, max_iter=10000),
    "Polynomial Regression Degree 2": Pipeline([
        ("poly", PolynomialFeatures(degree=2, include_bias=False)),
        ("linear", LinearRegression()),
    ]),
}

results = []

for name, model in models.items():
    pipe = Pipeline([
        ("preprocess", preprocess),
        ("model", model),
    ])
    pipe.fit(X_train, y_train)
    pred = pipe.predict(X_test)
    row = {"Model": name}
    row.update(regression_metrics(y_test, pred))
    results.append(row)

results_df = pd.DataFrame(results).sort_values(by="RMSE")
print(results_df.round(4))

best_model_name = results_df.iloc[0]["Model"]
print("\nConclusion:")
print("Best model by lowest RMSE:", best_model_name)
print("Use MAE when you want average absolute error.")
print("Use RMSE when large errors should be punished more.")
print("Use R2 to see how much variance is explained.")

# %%
# Tune Ridge, Lasso, ElasticNet using GridSearchCV.
tune_models = {
    "Ridge": (Ridge(random_state=42), {"model__alpha": [0.001, 0.01, 0.1, 1, 10, 100]}),
    "Lasso": (Lasso(random_state=42, max_iter=10000), {"model__alpha": [0.001, 0.01, 0.1, 1, 10]}),
    "ElasticNet": (ElasticNet(random_state=42, max_iter=10000), {
        "model__alpha": [0.001, 0.01, 0.1, 1],
        "model__l1_ratio": [0.1, 0.5, 0.9],
    }),
}

for name, (model, params) in tune_models.items():
    pipe = Pipeline([
        ("preprocess", preprocess),
        ("model", model),
    ])
    grid = GridSearchCV(pipe, params, cv=5, scoring="neg_root_mean_squared_error")
    grid.fit(X_train, y_train)
    print("\n", name)
    print("Best params:", grid.best_params_)
    print("Best CV RMSE:", -grid.best_score_)

```

10 Lab 8: Air Quality: AQI regression and SVM classification

```

# %%
# Lab target:
# Download Air Quality dataset from Kaggle.
# Predict Air Quality Index using Linear Regression.
# Classify AQI into five categories using SVM.
# Very Good, Good, Moderate, Poor, Worst.

import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression, Ridge

```

```

from sklearn.svm import SVC
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix, classification_report

# %%
def make_demo_air_quality(n=600):
    rng = np.random.default_rng(42)
    df = pd.DataFrame({
        "PM2.5": rng.normal(55, 25, n).clip(5, 250),
        "PM10": rng.normal(110, 45, n).clip(10, 400),
        "NO2": rng.normal(35, 18, n).clip(1, 150),
        "SO2": rng.normal(12, 8, n).clip(1, 80),
        "CO": rng.normal(1.2, 0.6, n).clip(0.1, 8),
        "O3": rng.normal(35, 15, n).clip(1, 180),
        "Temperature": rng.normal(28, 6, n),
        "Humidity": rng.normal(60, 20, n).clip(5, 100),
    })
    noise = rng.normal(0, 15, n)
    df["AQI"] = (
        0.85 * df["PM2.5"] + 0.45 * df["PM10"] +
        0.60 * df["NO2"] + 3.0 * df["CO"] + noise
    ).clip(0, 500)
    return df.round(2)

def aqi_category(aqi):
    if aqi <= 50:
        return "Very Good"
    elif aqi <= 100:
        return "Good"
    elif aqi <= 200:
        return "Moderate"
    elif aqi <= 300:
        return "Poor"
    else:
        return "Worst"

# %%
possible_files = ["AirQuality.csv", "air_quality.csv", "city_day.csv", "airquality.csv"]
csv_path = next((p for p in possible_files if os.path.exists(p)), None)

if csv_path:
    df = pd.read_csv(csv_path)
else:
    df = make_demo_air_quality()

print(df.head())
print(df.info())
print(df.isna().sum())

# %%
# Clean numeric columns.
for col in df.columns:
    df[col] = pd.to_numeric(df[col], errors="ignore")

# Choose AQI target. If AQI not available, use the last numeric column.
if "AQI" in df.columns:
    target_col = "AQI"
else:
    numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
    target_col = numeric_cols[-1]

# Fill missing numeric values.
num_cols = df.select_dtypes(include=[np.number]).columns.tolist()
df[num_cols] = df[num_cols].fillna(df[num_cols].median())

X = df[num_cols].drop(columns=[target_col], errors="ignore")
y = df[target_col]

print("Features:", X.columns.tolist())
print("Target:", target_col)

# %%
# Linear regression for AQI prediction.
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

reg_model = Pipeline([
    ("scaler", StandardScaler()),
    ("linear", LinearRegression()),
])

reg_model.fit(X_train, y_train)
y_pred = reg_model.predict(X_test)

print("Regression Results")
print("MAE:", mean_absolute_error(y_test, y_pred))

```

```

print("MSE:", mean_squared_error(y_test, y_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))
print("R2:", r2_score(y_test, y_pred))

plt.figure(figsize=(6, 5))
plt.scatter(y_test, y_pred, alpha=0.6)
plt.xlabel("Actual AQI")
plt.ylabel("Predicted AQI")
plt.title("Actual vs Predicted AQI")
plt.grid(True)
plt.show()

# %%
# Convert AQI into five classes and train SVM.
df["AQI_Category"] = df[target_col].apply(aqi_category)
print(df["AQI_Category"].value_counts())

X_cls = X.copy()
y_cls_text = df["AQI_Category"]

le = LabelEncoder()
y_cls = le.fit_transform(y_cls_text)

X_train, X_test, y_train, y_test = train_test_split(
    X_cls, y_cls, test_size=0.2, random_state=42, stratify=y_cls
)

svm_model = Pipeline([
    ("scaler", StandardScaler()),
    ("svm", SVC(kernel="rbf", C=10, gamma="scale", probability=True, random_state=42)),
])

svm_model.fit(X_train, y_train)
y_pred = svm_model.predict(X_test)

print("SVM Classification Results")
print("Accuracy:", accuracy_score(y_test, y_pred))
print("F1 macro:", f1_score(y_test, y_pred, average="macro"))
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("Classification Report:")
print(classification_report(y_test, y_pred, target_names=le.classes_))

```

11 Lab 9: Time Series: ACF, PACF, smoothing, ARMA, ARIMA

```

# %%
# Lab target:
# 1. Explain/use autocorrelation and calculate it on a dataset.
# 2. Apply moving average, exponential smoothing, Holt, Holt-Winters.
# 3. Explain and implement ARMA and ARIMA model.
# ARMA is for stationary series. ARIMA includes differencing for non-stationary series.

import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.metrics import mean_absolute_error, mean_squared_error
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.holtwinters import SimpleExpSmoothing, Holt, ExponentialSmoothing
from statsmodels.tsa.arima.model import ARIMA

# %%
def make_demo_time_series(periods=96):
    rng = np.random.default_rng(42)
    dates = pd.date_range("2018-01-01", periods=periods, freq="M")
    trend = np.linspace(100, 220, periods)
    season = 20 * np.sin(np.arange(periods) * 2 * np.pi / 12)
    noise = rng.normal(0, 8, periods)
    sales = trend + season + noise
    return pd.DataFrame({"Date": dates, "Sales": sales.round(2)})

# %%
CSV_PATH = "time_series.csv"
DATE_COL = "Date"
VALUE_COL = "Sales"

if os.path.exists(CSV_PATH):
    df = pd.read_csv(CSV_PATH)
else:
    df = make_demo_time_series()

# Auto-detect columns if needed.

```

```

if DATE_COL not in df.columns:
    DATE_COL = df.columns[0]
if VALUE_COL not in df.columns:
    numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
    VALUE_COL = numeric_cols[0]

df[DATE_COL] = pd.to_datetime(df[DATE_COL])
df = df.sort_values(DATE_COL).set_index(DATE_COL)
y = df[VALUE_COL].astype(float).asfreq(pd.infer_freq(df.index) or "M")
y = y.fillna(method="ffill")

print(y.head())
print("Frequency:", y.index.freq)

# %%
plt.figure(figsize=(10, 4))
plt.plot(y)
plt.title("Time Series")
plt.xlabel("Date")
plt.ylabel(VALUE_COL)
plt.grid(True)
plt.show()

# %%
# Autocorrelation calculation.
print("Autocorrelation values:")
for lag in range(1, 13):
    print(f"Lag {lag}:", round(y.autocorr(lag=lag), 4))

fig = plot_acf(y, lags=24)
plt.title("ACF - Autocorrelation Function")
plt.show()

fig = plot_pacf(y, lags=24, method="ywm")
plt.title("PACF - Partial Autocorrelation Function")
plt.show()

# %%
# ADF test for stationarity.
adf_stat, p_value, *_ = adfuller(y.dropna())
print("ADF statistic:", adf_stat)
print("ADF p-value:", p_value)
if p_value < 0.05:
    print("Series is likely stationary.")
else:
    print("Series is likely non-stationary. ARIMA differencing may be needed.")

# %%
# Train-test split by time order.
test_size = 12
train = y.iloc[:-test_size]
test = y.iloc[-test_size:]

# Moving average forecast: repeat last rolling mean.
window = 3
ma_value = train.rolling(window).mean().iloc[-1]
ma_forecast = pd.Series([ma_value] * len(test), index=test.index)

# Simple exponential smoothing.
ses_model = SimpleExpSmoothing(train).fit(optimized=True)
ses_forecast = ses_model.forecast(len(test))

# Holt trend method.
holt_model = Holt(train).fit(optimized=True)
holt_forecast = holt_model.forecast(len(test))

# Holt-Winters for trend and seasonality.
hw_model = ExponentialSmoothing(
    train, trend="add", seasonal="add", seasonal_periods=12
).fit(optimized=True)
hw_forecast = hw_model.forecast(len(test))

# ARMA via ARIMA with d=0.
arma_model = ARIMA(train, order=(2, 0, 2)).fit()
arma_forecast = arma_model.forecast(len(test))

# ARIMA with differencing.
arima_model = ARIMA(train, order=(2, 1, 2)).fit()
arima_forecast = arima_model.forecast(len(test))

# %%
def ts_metrics(name, actual, pred):
    return {
        "Model": name,
        "MAE": mean_absolute_error(actual, pred),
        "MSE": mean_squared_error(actual, pred),
        "RMSE": np.sqrt(mean_squared_error(actual, pred)),
        "MAPE": np.mean(np.abs((actual - pred) / actual)) * 100,
    }

```

```

results = pd.DataFrame([
    ts_metrics("Moving Average", test, ma_forecast),
    ts_metrics("Simple Exp Smoothing", test, ses_forecast),
    ts_metrics("Holt Trend", test, holt_forecast),
    ts_metrics("Holt-Winters", test, hw_forecast),
    ts_metrics("ARMA(2,0,2)", test, arma_forecast),
    ts_metrics("ARIMA(2,1,2)", test, arima_forecast),
]).sort_values("RMSE")

print(results.round(3))

# %%
plt.figure(figsize=(10, 5))
plt.plot(train.index, train, label="Train")
plt.plot(test.index, test, label="Actual Test")
plt.plot(test.index, hw_forecast, label="Holt-Winters")
plt.plot(test.index, arima_forecast, label="ARIMA")
plt.title("Time Series Forecasting")
plt.legend()
plt.grid(True)
plt.show()

```

12 Lab 10: Recommendation Systems: CF, factorization, metrics

```

# %%
# Lab target:
# Recommendation Systems:
# - Data collection and storage
# - Data filtering
# - Collaborative filtering
# - Factorization methods
# - Evaluation metrics: Recall, Precision, RMSE, MRR, MAP@K, NDCG

import os
import numpy as np
import pandas as pd

from sklearn.metrics.pairwise import cosine_similarity
from sklearn.decomposition import TruncatedSVD, NMF
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split

# %%
def make_demo_ratings(n_users=50, n_items=30, n_ratings=500):
    rng = np.random.default_rng(42)
    return pd.DataFrame({
        "user_id": rng.integers(1, n_users + 1, n_ratings),
        "item_id": rng.integers(1, n_items + 1, n_ratings),
        "rating": rng.integers(1, 6, n_ratings),
    }).drop_duplicates(subset=["user_id", "item_id"])

CSV_PATH = "ratings.csv"
if os.path.exists(CSV_PATH):
    ratings = pd.read_csv(CSV_PATH)
else:
    ratings = make_demo_ratings()

# Required columns: user_id, item_id, rating.
ratings = ratings[["user_id", "item_id", "rating"]].copy()
print(ratings.head())
print(ratings.shape)

# %%
# Popularity-based recommendation.
popularity = (
    ratings.groupby("item_id")
    .agg(avg_rating=("rating", "mean"), rating_count=("rating", "count"))
    .sort_values(["rating_count", "avg_rating"], ascending=False)
)
print("Top popular items:")
print(popularity.head(10))

# %%
# Train-test split by rating rows.
train_df, test_df = train_test_split(ratings, test_size=0.2, random_state=42)

user_item = train_df.pivot_table(
    index="user_id", columns="item_id", values="rating", fill_value=0
)

print(user_item.head())

# %%

```

```

# User-based collaborative filtering using cosine similarity.
user_similarity = cosine_similarity(user_item)
user_similarity_df = pd.DataFrame(
    user_similarity,
    index=user_item.index,
    columns=user_item.index,
)

def recommend_user_based(user_id, k=5):
    if user_id not in user_item.index:
        return popularity.index[:k].tolist()

    sim_scores = user_similarity_df[user_id].drop(index=user_id).sort_values(ascending=False)
    similar_users = sim_scores.head(5).index
    similar_ratings = user_item.loc[similar_users]

    scores = similar_ratings.mean(axis=0)
    already_rated = user_item.loc[user_id]
    scores = scores[already_rated == 0]
    return scores.sort_values(ascending=False).head(k).index.tolist()

print("User-based recommendations:", recommend_user_based(user_item.index[0], k=5))

# %%
# Item-based collaborative filtering using cosine similarity.
item_user = user_item.T
item_similarity = cosine_similarity(item_user)
item_similarity_df = pd.DataFrame(
    item_similarity,
    index=item_user.index,
    columns=item_user.index,
)

def recommend_item_based(user_id, k=5):
    if user_id not in user_item.index:
        return popularity.index[:k].tolist()

    user_ratings = user_item.loc[user_id]
    rated_items = user_ratings[user_ratings > 0].index
    scores = pd.Series(0.0, index=user_item.columns)

    for item in rated_items:
        scores += item_similarity_df[item] * user_ratings[item]

    scores = scores.drop(index=rated_items, errors="ignore")
    return scores.sort_values(ascending=False).head(k).index.tolist()

print("Item-based recommendations:", recommend_item_based(user_item.index[0], k=5))

# %%
# Matrix factorization using TruncatedSVD.
svd = TruncatedSVD(n_components=10, random_state=42)
user_factors = svd.fit_transform(user_item)
item_factors = svd.components_
pred_matrix = np.dot(user_factors, item_factors)

pred_df = pd.DataFrame(pred_matrix, index=user_item.index, columns=user_item.columns)

def recommend_svd(user_id, k=5):
    if user_id not in pred_df.index:
        return popularity.index[:k].tolist()
    scores = pred_df.loc[user_id].copy()
    already_rated = user_item.loc[user_id]
    scores = scores[already_rated == 0]
    return scores.sort_values(ascending=False).head(k).index.tolist()

print("SVD recommendations:", recommend_svd(user_item.index[0], k=5))

# %%
# Recommendation evaluation functions.
def precision_at_k(recommended, relevant, k):
    recommended = recommended[:k]
    if k == 0:
        return 0
    return len(set(recommended) & set(relevant)) / k

def recall_at_k(recommended, relevant, k):
    recommended = recommended[:k]
    if len(relevant) == 0:
        return 0
    return len(set(recommended) & set(relevant)) / len(relevant)

def average_precision_at_k(recommended, relevant, k):
    score = 0.0

```

```

hits = 0
for i, item in enumerate(recommended[:k], start=1):
    if item in relevant:
        hits += 1
    score += hits / i
return score / min(len(relevant), k) if relevant else 0

def reciprocal_rank_at_k(recommended, relevant, k):
    for i, item in enumerate(recommended[:k], start=1):
        if item in relevant:
            return 1 / i
    return 0

def dcg_at_k(recommended, relevant, k):
    score = 0.0
    for i, item in enumerate(recommended[:k], start=1):
        rel = 1 if item in relevant else 0
        score += rel / np.log2(i + 1)
    return score

def ndcg_at_k(recommended, relevant, k):
    ideal_recommended = list(relevant)[:k]
    ideal_dcg = dcg_at_k(ideal_recommended, relevant, k)
    if ideal_dcg == 0:
        return 0
    return dcg_at_k(recommended, relevant, k) / ideal_dcg

# %%
# Build relevant test items: rating >= 4 is considered relevant.
K = 5
metrics = []
for user_id in test_df["user_id"].unique():
    relevant = test_df[(test_df["user_id"] == user_id) & (test_df["rating"] >= 4)]["item_id"].tolist()
    if not relevant:
        continue
    recs = recommend_svd(user_id, k=K)
    metrics.append({
        "user_id": user_id,
        "precision@K": precision_at_k(recs, relevant, K),
        "recall@K": recall_at_k(recs, relevant, K),
        "AP@K": average_precision_at_k(recs, relevant, K),
        "RR@K": reciprocal_rank_at_k(recs, relevant, K),
        "NDCG@K": ndcg_at_k(recs, relevant, K),
    })

metrics_df = pd.DataFrame(metrics)
print(metrics_df.mean(numeric_only=True))
print("MAP@K:", metrics_df["AP@K"].mean())
print("MRR@K:", metrics_df["RR@K"].mean())

# %%
# RMSE for known test ratings where prediction exists.
true_ratings = []
pred_ratings = []
for _, row in test_df.iterrows():
    u, i, r = row["user_id"], row["item_id"], row["rating"]
    if u in pred_df.index and i in pred_df.columns:
        true_ratings.append(r)
        pred_ratings.append(pred_df.loc[u, i])

if true_ratings:
    print("Rating prediction RMSE:", np.sqrt(mean_squared_error(true_ratings, pred_ratings)))

```

13 Lab 11: TensorFlow/Keras: activations and single-layer NN

```

# %%
# Lab target:
# Explore TensorFlow and Keras libraries.
# Implement activation functions on datasets in Jupyter Notebook.
# Build a single-layer neural network and compare with a small hidden-layer model.

# If needed:
# python -m pip install tensorflow

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

```



```

from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

print("TensorFlow version:", tf.__version__)

# %%
# Plot activation functions.
x = np.linspace(-6, 6, 400)
sigmoid = 1 / (1 + np.exp(-x))
tanh = np.tanh(x)
relu = np.maximum(0, x)

plt.figure(figsize=(8, 4))
plt.plot(x, sigmoid, label="Sigmoid")
plt.plot(x, tanh, label="Tanh")
plt.plot(x, relu, label="ReLU")
plt.title("Activation Functions")
plt.xlabel("x")
plt.ylabel("activation(x)")
plt.legend()
plt.grid(True)
plt.show()

# %%
# Dataset.
X, y = make_moons(n_samples=1000, noise=0.25, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

plt.figure(figsize=(6, 5))
plt.scatter(X[:, 0], X[:, 1], c=y, s=20)
plt.title("Moons Dataset")
plt.show()

# %%
# Single-layer neural network: no hidden layer.
single_layer_model = keras.Sequential([
    layers.Input(shape=(2,)),
    layers.Dense(1, activation="sigmoid"),
])

single_layer_model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"],
)

history_single = single_layer_model.fit(
    X_train, y_train,
    validation_split=0.2,
    epochs=50,
    batch_size=32,
    verbose=0,
)

pred_prob = single_layer_model.predict(X_test).ravel()
pred = (pred_prob >= 0.5).astype(int)
print("Single-layer accuracy:", accuracy_score(y_test, pred))
print(classification_report(y_test, pred))

# %%
# Non-linear model with hidden layers.
mlp_model = keras.Sequential([
    layers.Input(shape=(2,)),
    layers.Dense(16, activation="relu"),
    layers.Dense(8, activation="relu"),
    layers.Dense(1, activation="sigmoid"),
])

mlp_model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"],
)

history_mlp = mlp_model.fit(
    X_train, y_train,
    validation_split=0.2,
    epochs=50,
    batch_size=32,
    verbose=0,
)

```

```

)

pred_prob = mlp_model.predict(X_test).ravel()
pred = (pred_prob >= 0.5).astype(int)
print("MLP accuracy:", accuracy_score(y_test, pred))
print(classification_report(y_test, pred))

# %%
plt.figure(figsize=(8, 4))
plt.plot(history_single.history["val_accuracy"], label="Single-layer val accuracy")
plt.plot(history_mlp.history["val_accuracy"], label="MLP val accuracy")
plt.xlabel("Epoch")
plt.ylabel("Validation Accuracy")
plt.title("Single-layer vs Hidden-layer Neural Network")
plt.legend()
plt.grid(True)
plt.show()

```

14 Lab 12: Autoencoder reconstruction

```

# %%
# Lab target:
# Introduction to Autoencoders.
# Autoencoders learn compressed representations and reconstruct input.

# If needed:
# python -m pip install tensorflow

import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# %%
digits = load_digits()
X = digits.data.astype("float32")

scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test = train_test_split(X_scaled, test_size=0.2, random_state=42)

input_dim = X_train.shape[1]
encoding_dim = 16

# %%
input_layer = keras.Input(shape=(input_dim,))
encoded = layers.Dense(32, activation="relu")(input_layer)
encoded = layers.Dense(encoding_dim, activation="relu")(encoded)
decoded = layers.Dense(32, activation="relu")(encoded)
decoded = layers.Dense(input_dim, activation="sigmoid")(decoded)

autoencoder = keras.Model(input_layer, decoded)
encoder = keras.Model(input_layer, encoded)

autoencoder.compile(optimizer="adam", loss="mse")

autoencoder.summary()

# %%
history = autoencoder.fit(
    X_train, X_train,
    validation_split=0.2,
    epochs=50,
    batch_size=32,
    verbose=0,
)

reconstructed = autoencoder.predict(X_test)
encoded_features = encoder.predict(X_test)

reconstruction_error = np.mean((X_test - reconstructed) ** 2, axis=1)
print("Mean reconstruction error:", reconstruction_error.mean())
print("Encoded feature shape:", encoded_features.shape)

# %%
plt.figure(figsize=(8, 4))
plt.plot(history.history["loss"], label="Train loss")

```

```

plt.plot(history.history["val_loss"], label="Validation loss")
plt.xlabel("Epoch")
plt.ylabel("MSE Loss")
plt.title("Autoencoder Training")
plt.legend()
plt.grid(True)
plt.show()

# %%
# Show original vs reconstructed digit images.
n = 5
plt.figure(figsize=(10, 4))
for i in range(n):
    plt.subplot(2, n, i + 1)
    plt.imshow(X_test[i].reshape(8, 8), cmap="gray")
    plt.title("Original")
    plt.axis("off")

    plt.subplot(2, n, i + 1 + n)
    plt.imshow(reconstructed[i].reshape(8, 8), cmap="gray")
    plt.title("Reconstructed")
    plt.axis("off")
plt.show()

```

15 Lab 13: Backprop, weight update, Early Stopping, Dropout, L1/L2

```

# %%
# Lab target:
# Deep Learning Essentials:
# - Update weights with a single training set element.
# - Early stopping for preventing overfitting.
# - Dropout.
# - L1 and L2 regularization.

import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, regularizers

# %%
# Manual single training example weight update for one sigmoid neuron.
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

x = np.array([0.5, -1.2, 0.3])
y_true = 1

w = np.array([0.1, -0.2, 0.05])
b = 0.0
learning_rate = 0.1

# Forward pass.
z = np.dot(w, x) + b
y_pred = sigmoid(z)
loss = -(y_true * np.log(y_pred) + (1 - y_true) * np.log(1 - y_pred))

# Backprop for binary cross entropy with sigmoid.
dz = y_pred - y_true
dw = dz * x
db = dz

# Update.
w_new = w - learning_rate * dw
b_new = b - learning_rate * db

print("Before update")
print("z:", z, "prediction:", y_pred, "loss:", loss)
print("w:", w, "b:", b)
print("Gradients dw:", dw, "db:", db)
print("After update")
print("w_new:", w_new, "b_new:", b_new)

# %%
# Dataset for regularization demo.
X, y = make_classification(
    n_samples=1500,

```

```

n_features=30,
n_informative=8,
n_redundant=8,
random_state=42,
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# %%
def build_model(use_dropout=False, use_l1_l2=False):
    reg = regularizers.l1_l2(l1=1e-5, l2=1e-4) if use_l1_l2 else None
    model = keras.Sequential()
    model.add(layers.Input(shape=(X_train.shape[1],)))
    model.add(layers.Dense(128, activation="relu", kernel_regularizer=reg))
    if use_dropout:
        model.add(layers.Dropout(0.4))
    model.add(layers.Dense(64, activation="relu", kernel_regularizer=reg))
    if use_dropout:
        model.add(layers.Dropout(0.3))
    model.add(layers.Dense(1, activation="sigmoid"))
    model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
    return model

callbacks = [
    keras.callbacks.EarlyStopping(
        monitor="val_loss",
        patience=8,
        restore_best_weights=True,
    )
]

model_plain = build_model(use_dropout=False, use_l1_l2=False)
history_plain = model_plain.fit(
    X_train, y_train,
    validation_split=0.2,
    epochs=100,
    batch_size=32,
    verbose=0,
)

model_regularized = build_model(use_dropout=True, use_l1_l2=True)
history_regularized = model_regularized.fit(
    X_train, y_train,
    validation_split=0.2,
    epochs=100,
    batch_size=32,
    callbacks=callbacks,
    verbose=0,
)

print("Plain model test:", model_plain.evaluate(X_test, y_test, verbose=0))
print("Regularized model test:", model_regularized.evaluate(X_test, y_test, verbose=0))

# %%
plt.figure(figsize=(8, 4))
plt.plot(history_plain.history["val_loss"], label="Plain validation loss")
plt.plot(history_regularized.history["val_loss"], label="Regularized validation loss")
plt.xlabel("Epoch")
plt.ylabel("Validation loss")
plt.title("Early Stopping + Dropout + L1/L2 Regularization")
plt.legend()
plt.grid(True)
plt.show()

```

16 Lab 14: Batch, Mini-batch, SGD, Momentum, Adam, gradients

```

# %%
# Lab target:
# Batch training, mini-batch training, stochastic gradient descent.
# Implement different optimizers: classic SGD, Momentum, Adam.
# Implement/visualize gradient problems: vanishing and exploding gradients.

import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

```

```

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# %%
X, y = make_classification(
    n_samples=2000,
    n_features=20,
    n_informative=10,
    n_redundant=5,
    random_state=42,
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# %%
def create_model(optimizer):
    model = keras.Sequential([
        layers.Input(shape=(X_train.shape[1],)),
        layers.Dense(64, activation="relu"),
        layers.Dense(32, activation="relu"),
        layers.Dense(1, activation="sigmoid"),
    ])
    model.compile(optimizer=optimizer, loss="binary_crossentropy", metrics=["accuracy"])
    return model

# Batch size options.
batch_settings = {
    "Full Batch": len(X_train),
    "Mini Batch 32": 32,
    "Stochastic Batch 1": 1,
}

histories = {}
for name, batch_size in batch_settings.items():
    model = create_model(keras.optimizers.SGD(learning_rate=0.01))
    histories[name] = model.fit(
        X_train, y_train,
        validation_split=0.2,
        epochs=25,
        batch_size=batch_size,
        verbose=0,
    )
    print(name, "test:", model.evaluate(X_test, y_test, verbose=0))

# %%
plt.figure(figsize=(8, 4))
for name, hist in histories.items():
    plt.plot(hist.history["val_loss"], label=name)
plt.xlabel("Epoch")
plt.ylabel("Validation loss")
plt.title("Batch vs Mini-batch vs SGD")
plt.legend()
plt.grid(True)
plt.show()

# %%
# Compare optimizers: SGD, Momentum, Adam.
optimizer_settings = {
    "SGD": keras.optimizers.SGD(learning_rate=0.01),
    "Momentum": keras.optimizers.SGD(learning_rate=0.01, momentum=0.9),
    "Adam": keras.optimizers.Adam(learning_rate=0.001),
}

opt_histories = {}
for name, opt in optimizer_settings.items():
    model = create_model(opt)
    opt_histories[name] = model.fit(
        X_train, y_train,
        validation_split=0.2,
        epochs=30,
        batch_size=32,
        verbose=0,
    )
    print(name, "test:", model.evaluate(X_test, y_test, verbose=0))

plt.figure(figsize=(8, 4))
for name, hist in opt_histories.items():
    plt.plot(hist.history["val_loss"], label=name)
plt.xlabel("Epoch")
plt.ylabel("Validation loss")
plt.title("Optimizer Comparison")

```

```

plt.legend()
plt.grid(True)
plt.show()

# %%
# Vanishing gradient demonstration with sigmoid derivative.
z = np.linspace(-10, 10, 500)
sigmoid = 1 / (1 + np.exp(-z))
sigmoid_grad = sigmoid * (1 - sigmoid)
relu_grad = (z > 0).astype(float)

plt.figure(figsize=(8, 4))
plt.plot(z, sigmoid_grad, label="Sigmoid gradient")
plt.plot(z, relu_grad, label="ReLU gradient")
plt.xlabel("z")
plt.ylabel("gradient")
plt.title("Gradient Problem: Sigmoid Saturation vs ReLU")
plt.legend()
plt.grid(True)
plt.show()

# %%
# Exploding gradient demonstration using repeated multiplication.
values = []
grad = 1.0
for step in range(1, 31):
    grad *= 1.5
    values.append(grad)

plt.figure(figsize=(8, 4))
plt.plot(range(1, 31), values, marker="o")
plt.xlabel("Step")
plt.ylabel("Gradient magnitude")
plt.title("Exploding Gradient Demonstration")
plt.grid(True)
plt.show()

```

17 Lab 15: PyTorch CNN and data augmentation

```

# %%
# Lab target:
# Convolutional Neural Network using PyTorch.
# Explore PyTorch library, convolution concept, and data augmentation.
# If needed:
# python -m pip install torch torchvision

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

print("PyTorch version:", torch.__version__)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)

# %%
# Data augmentation and dataset.
# FakeData is used so the lab runs without downloading a dataset.
# Replace FakeData with datasets.ImageFolder("data/train", transform=train_transform) for real images.

train_transform = transforms.Compose([
    transforms.Resize((64, 64)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(10),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5, 0.5, 0.5], std=[0.5, 0.5, 0.5]),
])

test_transform = transforms.Compose([
    transforms.Resize((64, 64)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5, 0.5, 0.5], std=[0.5, 0.5, 0.5]),
])

train_dataset = datasets.FakeData(
    size=500,
    image_size=(3, 64, 64),
    num_classes=2,
    transform=train_transform,
)

test_dataset = datasets.FakeData(
    size=100,

```

```

    image_size=(3, 64, 64),
    num_classes=2,
    transform=test_transform,
)

train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)

# %%
# CNN model.
class SimpleCNN(nn.Module):
    def __init__(self, num_classes=2):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 16, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(16, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64 * 8 * 8, 128),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(128, num_classes),
        )

    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)
        return x

model = SimpleCNN(num_classes=2).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

print(model)

# %%
# Training loop.
def train_one_epoch(model, loader):
    model.train()
    total_loss = 0
    correct = 0
    total = 0
    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        total_loss += loss.item() * images.size(0)
        preds = outputs.argmax(dim=1)
        correct += (preds == labels).sum().item()
        total += labels.size(0)
    return total_loss / total, correct / total

def evaluate(model, loader):
    model.eval()
    total_loss = 0
    correct = 0
    total = 0
    with torch.no_grad():
        for images, labels in loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss = criterion(outputs, labels)
            total_loss += loss.item() * images.size(0)
            preds = outputs.argmax(dim=1)
            correct += (preds == labels).sum().item()
            total += labels.size(0)
    return total_loss / total, correct / total

for epoch in range(3):
    train_loss, train_acc = train_one_epoch(model, train_loader)
    test_loss, test_acc = evaluate(model, test_loader)
    print(
        f"Epoch {epoch + 1}: "
        f"train_loss={train_loss:.4f}, train_acc={train_acc:.4f}, "
        f"test_loss={test_loss:.4f}, test_acc={test_acc:.4f}"
    )

```

```
# %%
# Save model.
torch.save(model.state_dict(), "simple_cnn_pytorch.pth")
print("Saved model to simple_cnn_pytorch.pth")
```

18 Lab 16: Transfer Learning with Inception Network

```
# %%
# Lab target:
# Transfer Learning and Inception Network as an example.
# This script uses torchvision InceptionV3. For a real project, use ImageFolder data.
# If internet is available and you want pretrained weights, use weights="DEFAULT".

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms, models

print("PyTorch version:", torch.__version__)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)

# %%
num_classes = 2
batch_size = 8

transform = transforms.Compose([
    transforms.Resize((299, 299)),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])

train_dataset = datasets.FakeData(
    size=80,
    image_size=(3, 299, 299),
    num_classes=num_classes,
    transform=transform,
)

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)

# %%
# InceptionV3 transfer learning setup.
# weights=None avoids downloading weights and keeps this lab offline-executable.
# For real transfer learning: models.inception_v3(weights=models.Inception_V3_Weights.DEFAULT)
model = models.inception_v3(weights=None, aux_logits=True)

# Freeze feature extractor layers if using pretrained weights.
for param in model.parameters():
    param.requires_grad = False

# Replace final classifier and auxiliary classifier.
model.fc = nn.Linear(model.fc.in_features, num_classes)
model.AuxLogits.fc = nn.Linear(model.AuxLogits.fc.in_features, num_classes)

# Train only new heads.
for param in model.fc.parameters():
    param.requires_grad = True
for param in model.AuxLogits.fc.parameters():
    param.requires_grad = True

model = model.to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(filter(lambda p: p.requires_grad, model.parameters()), lr=0.001)

# %%
# Training loop for Inception. It may return output and auxiliary output during training.
model.train()
for epoch in range(1):
    total_loss = 0
    correct = 0
    total = 0
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()

        outputs = model(images)
        if hasattr(outputs, "logits"):
            logits = outputs.logits
            aux_logits = outputs.aux_logits
            loss = criterion(logits, labels) + 0.4 * criterion(aux_logits, labels)
```



```

else:
    logits = outputs
    loss = criterion(logits, labels)

    loss.backward()
    optimizer.step()

    total_loss += loss.item() * images.size(0)
    preds = logits.argmax(dim=1)
    correct += (preds == labels).sum().item()
    total += labels.size(0)

print(f"Epoch {epoch + 1}: loss={total_loss / total:.4f}, acc={correct / total:.4f}")

# %%
# Inference mode.
model.eval()
with torch.no_grad():
    images, labels = next(iter(train_loader))
    images = images.to(device)
    outputs = model(images)
    if hasattr(outputs, "logits"):
        outputs = outputs.logits
    predictions = outputs.argmax(dim=1).cpu().numpy()
    print("Predictions:", predictions[:10])

```

19 Lab 17: YOLO object detection with Ultralytics

```

# %%
# Lab target:
# Implement YOLO algorithm at high level using Ultralytics YOLO.
# Object detection is harder than image classification because it predicts:
# 1. class label
# 2. bounding box coordinates
# 3. confidence score
#
# If needed:
# python -m pip install ultralytics

from pathlib import Path

try:
    from ultralytics import YOLO
except ImportError:
    raise SystemExit("Install ultralytics first: python -m pip install ultralytics")

# %%
# Load model.
# yolov8n.pt may download weights if not already present.
# For offline use, keep yolov8n.pt in the current folder and pass that path.
model = YOLO("yolov8n.pt")

# %%
# Predict on an image or folder.
# Replace with your own image path.
SOURCE = "sample_image.jpg"
if not Path(SOURCE).exists():
    print("Put sample_image.jpg in the current folder to run prediction.")
else:
    results = model.predict(source=SOURCE, conf=0.25, save=True)
    for result in results:
        print("Boxes:", result.boxes)
        print("Class names:", result.names)

# %%
# Train YOLO on a custom dataset.
# Dataset folder should contain data.yaml with train/val paths and class names.
# Example data.yaml:
# train: images/train
# val: images/val
# nc: 2
# names: ["helmet", "no_helmet"]
#
# Uncomment after preparing dataset:
# train_results = model.train(data="data.yaml", epochs=10, imgsz=640, batch=8)

# %%
# Validate trained model.
# Uncomment after training:
# metrics = model.val(data="data.yaml")
# print(metrics)

# %%
# Export model.

```

```
# Uncomment if needed:
# model.export(format="onnx")
```

20 Lab 18: Bonus: RNN, LSTM, GRU sequence forecasting

```
# %%
# Bonus complete-syllabus lab:
# RNN, LSTM, GRU for sequence/time-series prediction.
# Use this if your later sessions include recurrent neural networks.

import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# %%
# Create sine-wave time series.
np.random.seed(42)
t = np.arange(0, 400)
series = np.sin(0.05 * t) + 0.3 * np.sin(0.15 * t) + np.random.normal(0, 0.05, len(t))
series = series.reshape(-1, 1)

scaler = MinMaxScaler()
series_scaled = scaler.fit_transform(series)

# %%
def make_sequences(data, window=20):
    X, y = [], []
    for i in range(len(data) - window):
        X.append(data[i:i + window])
        y.append(data[i + window])
    return np.array(X), np.array(y)

WINDOW = 20
X, y = make_sequences(series_scaled, WINDOW)

split = int(len(X) * 0.8)
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

print(X_train.shape, y_train.shape)

# %%
def build_sequence_model(cell_type="LSTM"):
    model = keras.Sequential()
    model.add(layers.Input(shape=(WINDOW, 1)))
    if cell_type == "RNN":
        model.add(layers.SimpleRNN(32))
    elif cell_type == "GRU":
        model.add(layers.GRU(32))
    else:
        model.add(layers.LSTM(32))
    model.add(layers.Dense(1))
    model.compile(optimizer="adam", loss="mse")
    return model

models = {}
histories = {}
for cell in ["RNN", "LSTM", "GRU"]:
    model = build_sequence_model(cell)
    histories[cell] = model.fit(
        X_train, y_train,
        validation_split=0.2,
        epochs=30,
        batch_size=16,
        verbose=0,
    )
    models[cell] = model
    pred = model.predict(X_test)
    rmse = np.sqrt(mean_squared_error(y_test, pred))
    print(cell, "scaled RMSE:", rmse)

# %%
plt.figure(figsize=(8, 4))
for cell, hist in histories.items():
    plt.plot(hist.history["val_loss"], label=cell)
plt.title("RNN vs LSTM vs GRU")
plt.xlabel("Epoch")
plt.ylabel("Validation loss")
```

```
plt.legend()
plt.grid(True)
plt.show()
```

21 Lab 19: Bonus: BERT text classification

```
# %%
# Bonus complete-syllabus lab:
# BERT/Transformer text classification using Hugging Face Transformers.
# Use this if your later sessions include transformers, BERT, or NLP fine-tuning.
# If needed:
# python -m pip install transformers datasets torch scikit-learn

import numpy as np
import torch
from torch.utils.data import Dataset, DataLoader
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, accuracy_score
from transformers import AutoTokenizer, AutoModelForSequenceClassification
from torch.optim import AdamW

# %%
texts = [
    "This product is excellent and very useful",
    "I hated the service and the quality was poor",
    "The movie was amazing",
    "The food was bad and cold",
    "I am happy with the result",
    "This is a terrible experience",
    "The course was helpful and clear",
    "The app crashes again and again",
]
labels = [1, 0, 1, 0, 1, 0, 1, 0]

train_texts, test_texts, train_labels, test_labels = train_test_split(
    texts, labels, test_size=0.25, random_state=42, stratify=labels
)

MODEL_NAME = "distilbert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)

# %%
class TextDataset(Dataset):
    def __init__(self, texts, labels, tokenizer, max_len=64):
        self.texts = texts
        self.labels = labels
        self.tokenizer = tokenizer
        self.max_len = max_len

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        enc = self.tokenizer(
            self.texts[idx],
            truncation=True,
            padding="max_length",
            max_length=self.max_len,
            return_tensors="pt",
        )
        item = {k: v.squeeze(0) for k, v in enc.items()}
        item["labels"] = torch.tensor(self.labels[idx], dtype=torch.long)
        return item

train_ds = TextDataset(train_texts, train_labels, tokenizer)
test_ds = TextDataset(test_texts, test_labels, tokenizer)

train_loader = DataLoader(train_ds, batch_size=2, shuffle=True)
test_loader = DataLoader(test_ds, batch_size=2)

# %%
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = AutoModelForSequenceClassification.from_pretrained(MODEL_NAME, num_labels=2)
model = model.to(device)
optimizer = AdamW(model.parameters(), lr=2e-5)

# %%
# Fine-tune for a small number of epochs.
model.train()
for epoch in range(1):
    total_loss = 0
    for batch in train_loader:
        batch = {k: v.to(device) for k, v in batch.items()}
        optimizer.zero_grad()
```

```

        outputs = model(**batch)
        loss = outputs.loss
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    print("Epoch", epoch + 1, "loss", total_loss / len(train_loader))

# %%
# Evaluate.
model.eval()
y_true, y_pred = [], []
with torch.no_grad():
    for batch in test_loader:
        labels = batch.pop("labels").to(device)
        batch = {k: v.to(device) for k, v in batch.items()}
        outputs = model(**batch)
        preds = outputs.logits.argmax(dim=1)
        y_true.extend(labels.cpu().numpy())
        y_pred.extend(preds.cpu().numpy())

print("Accuracy:", accuracy_score(y_true, y_pred))
print(classification_report(y_true, y_pred))

```

22 Lab 20: End-to-end sklearn pipeline and model saving

```

# %%
# Lab target:
# Complete ML pipeline for VS Code/Jupyter:
# load data, preprocess, train, evaluate, cross-validate, tune, save model.

import os
import joblib
import numpy as np
import pandas as pd

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV, cross_val_score
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix, f1_score, accuracy_score

# %%
CSV_PATH = "final_project_dataset.csv"
TARGET_COL = None

if os.path.exists(CSV_PATH):
    df = pd.read_csv(CSV_PATH)
    target_col = TARGET_COL or df.columns[-1]
else:
    data = load_breast_cancer(as_frame=True)
    df = data.frame.copy()
    target_col = "target"

X = df.drop(columns=[target_col])
y = df[target_col]

num_cols = X.select_dtypes(include=[np.number]).columns.tolist()
cat_cols = X.select_dtypes(exclude=[np.number]).columns.tolist()

preprocess = ColumnTransformer([
    ("num", Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
    ]), num_cols),
    ("cat", Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore")),
    ]), cat_cols),
])

model = RandomForestClassifier(random_state=42)

pipe = Pipeline([
    ("preprocess", preprocess),
    ("model", model),
])

# %%
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y if y.nunique() <= 20 else None
)

```

```

pipe.fit(X_train, y_train)
y_pred = pipe.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("F1 macro:", f1_score(y_test, y_pred, average="macro"))
print("Confusion matrix:")
print(confusion_matrix(y_test, y_pred))
print("Classification report:")
print(classification_report(y_test, y_pred))

# %%
# Cross-validation.
cv_scores = cross_val_score(pipe, X, y, cv=5, scoring="f1_macro")
print("CV F1 scores:", cv_scores)
print("Mean CV F1:", cv_scores.mean())

# %%
# Hyperparameter tuning.
params = {
    "model__n_estimators": [100, 200],
    "model__max_depth": [None, 5, 10],
    "model__min_samples_split": [2, 5],
}

grid = GridSearchCV(pipe, params, cv=5, scoring="f1_macro", n_jobs=-1)
grid.fit(X_train, y_train)

print("Best params:", grid.best_params_)
print("Best CV F1:", grid.best_score_)

best_model = grid.best_estimator_
y_pred = best_model.predict(X_test)
print("Tuned test F1:", f1_score(y_test, y_pred, average="macro"))

# %%
# Save final pipeline.
joblib.dump(best_model, "final_ml_pipeline.joblib")
print("Saved model as final_ml_pipeline.joblib")

# Load and predict again.
loaded_model = joblib.load("final_ml_pipeline.joblib")
print("Loaded model prediction sample:")
print(loaded_model.predict(X_test.head()))

```